Documentation SSH

Installation, sécurisation et transfert de fichiers sous Linux (Kali, Debian, Ubuntu) et Windows

[VOTRE NOM]
Licence 2 — Ingénierie informatique, parcours ARSB
Objectif : devenir expert en cybersécurité
Octobre 2026

Cadre : Ce document rassemble mes notes de cours et mes travaux pratiques sur SSH, réalisés dans mon home lab (VirtualBox, Kali Linux, Windows). Toutes les commandes sont à exécuter uniquement sur des machines que je possède ou avec une autorisation.

Sommaire

Sommaire 2

1. Introduction à SSH 3

1.1 Définition . . . 3

1.2 Fonctionnement d'une connexion . . . 3

1.3 Authentification par clés . . . 3

2. SSH sous Linux (Kali, Debian, Ubuntu) 4

2.1 Installer et vérifier le service . . . 4

2.2 Sauvegarder et modifier la configuration . . . 4

2.3 Créer et déployer une clé SSH . . . 4

2.4 Durcir la configuration (hardening) . . . 4

2.5 Valider et appliquer . . . 5

2.6 Fail2Ban : bannir les attaquants automatiquement . . . 5

2.7 Pare-feu UFW . . . 5

2.8 Lire les journaux . . . 6

2.9 Donner accès à plusieurs utilisateurs . . . 6

2.10 Erreurs fréquentes . . . 6

2.11 Travaux pratiques : durcir une VM Kali . . . 6

3. SSH sous Windows (OpenSSH) 7

3.1 Installer et démarrer le serveur . . . 7

3.2 Fichier de configuration . . . 7

3.3 Authentification par clé sous Windows . . . 7

3.4 Durcir la configuration . . . 8

3.5 Pare-feu Windows . . . 8

3.6 Surveillance et audit . . . 8

3.7 Dépannage . . . 8

4. Transférer des fichiers avec SSH 10

4.1 SCP (copie en une ligne) . . . 10

4.2 SFTP (session interactive) . . . 10

4.3 Scénario complet . . . 10

5. Aide-mémoire et bonnes pratiques 11

Checklist avant de terminer . . . 11

Glossaire . . . 11

1. Introduction à SSH

1.1 Définition

SSH (Secure Shell) est un protocole qui permet d'ouvrir une session à distance sur une machine à travers un canal **chiffré**. Il a remplacé Telnet, Rsh et Rlogin, qui envoyaient mots de passe et commandes en clair. SSH sert aussi à transférer des fichiers (SCP, SFTP) et à créer des tunnels réseau. Il écoute par défaut sur le port **TCP 22**.

Critère	Telnet	SSH
Chiffrement	Aucun (texte clair)	Oui (AES, ChaCha20)
Authentification	Mot de passe visible sur le réseau	Mot de passe ou clés cryptographiques
Intégrité des données	Aucune vérification	Vérifiée (MAC)
Port par défaut	23	22

1.2 Fonctionnement d'une connexion

- **Négociation** : client et serveur choisissent les algorithmes de chiffrement et d'échange de clés.
- **Authentification du serveur** : le serveur présente sa clé d'hôte. Le client la compare avec celle de `~/.ssh/known_hosts` pour détecter une attaque de l'homme du milieu (MITM).
- **Canal chiffré** : une clé de session éphémère est créée, toutes les données sont ensuite chiffrées.
- **Authentification de l'utilisateur** : par mot de passe, ou mieux, par paire de clés.

1.3 Authentification par clés

La **clé privée** reste sur le poste du client, protégée par une phrase de passe, et n'est jamais envoyée. La **clé publique** est copiée sur le serveur (fichier `authorized_keys`). Le serveur envoie un défi que seule la clé privée peut signer. On utilise l'algorithme **Ed25519**, plus moderne et plus robuste que RSA.

> **Règle d'or :** Ne désactive jamais l'authentification par mot de passe avant d'avoir **testé qu'une connexion par clé fonctionne**, et garde une session ouverte pendant tes tests. Sinon tu peux te bloquer hors du serveur.

2. SSH sous Linux (Kali, Debian, Ubuntu)

2.1 Installer et vérifier le service

```
sudo apt update
sudo apt install openssh-server -y

sudo systemctl status ssh        # état du service
sudo systemctl enable ssh        # démarrage automatique
sudo systemctl start ssh         # démarrer
sudo systemctl restart ssh       # redémarrer (après chaque modification)

sudo ss -lnpt | grep ssh         # vérifier le port d'écoute
```

Options de `ss` : **-l** en écoute, **-n** numérique, **-p** processus, **-t** TCP.

2.2 Sauvegarder et modifier la configuration

Le fichier du serveur est `/etc/ssh/sshd_config` (ne pas le confondre avec `ssh_config`, qui configure le client). Toujours faire une copie avant de modifier.

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
sudo nano /etc/ssh/sshd_config
```

2.3 Créer et déployer une clé SSH

Sur le poste **client** (par exemple ma machine Kali) :

```
ssh-keygen -t ed25519 -C "votre_email@exemple.com"
ssh-copy-id utilisateur@IP_DU_SERVEUR
ssh-copy-id -p 4422 utilisateur@IP_DU_SERVEUR     # si le port a été changé

ssh -i ~/.ssh/id_ed25519 utilisateur@IP_DU_SERVEUR   # test de connexion par clé
```

`ssh-copy-id` copie la clé publique dans `~/.ssh/authorized_keys` du serveur. Sur le serveur, les droits doivent être **700** pour `~/.ssh` et **600** pour `authorized_keys`.

2.4 Durcir la configuration (hardening)

Une fois la connexion par clé validée, on applique ces directives dans `sshd_config` :

```
PermitRootLogin no                # interdit la connexion directe de root
PubkeyAuthentication yes          # active les clés SSH
PasswordAuthentication no         # désactive les mots de passe
KbdInteractiveAuthentication no   # ancien nom : ChallengeResponseAuthentication
UsePAM yes                        # conserve la gestion Linux des comptes
Port 4422                         # port non standard (réduit le bruit des bots)
AllowUsers secuser                # seuls ces utilisateurs peuvent se connecter
MaxAuthTries 3                    # 3 essais maximum par connexion
LoginGraceTime 30                 # 30 s pour s'authentifier
X11Forwarding no                  # pas de redirection graphique
AllowTcpForwarding no             # pas de tunnels
AllowAgentForwarding no
ClientAliveInterval 90            # contrôle de présence du client toutes les 90 s
ClientAliveCountMax 1             # coupe après 1 contrôle sans réponse (environ 3 min)
```

Directive	Rôle et intérêt
`PermitRootLogin no`	Les attaquants connaissent le nom *root* : on l'interdit et on utilise *sudo* après connexion.

Directive	Rôle et intérêt
`PasswordAuthentication no`	Bloque les attaques par force brute sur les mots de passe : seule la clé fonctionne.
`AllowUsers / AllowGroups`	Principe du moindre privilège : liste blanche des comptes autorisés.
`Port`	Évite les scans automatiques du port 22. C'est un premier filtre, pas une protection suffisante seule.
`MaxAuthTries`	Limite les essais de connexion échoués.

Correction utile : `ClientAliveCountMax 0` ne coupe **pas** la session : cette valeur désactive la déconnexion. Pour une déconnexion automatique, utilise par exemple `ClientAliveInterval 90` avec `ClientAliveCountMax 1`.

Attention : Si tu changes le port, pense à l'ouvrir dans le pare-feu et à le déclarer dans Fail2Ban avant de redémarrer SSH. Puis connecte-toi avec `ssh -p 4422 utilisateur@IP`.

2.5 Valider et appliquer

```
sudo sshd -t                                    # vérifie la syntaxe (aucun message = OK)
sudo systemctl restart ssh                      # applique les changements
sudo systemctl status ssh                       # vérifie que le service tourne
sudo sshd -T | grep -E "passwordauthentication|permitrootlogin|^port"   # config effective
```

Conseil : ouvre **une deuxième fenêtre de terminal** et teste la nouvelle connexion avant de fermer la première. Si le test échoue, la première session te permet de réparer le fichier.

2.6 Fail2Ban : bannir les attaquants automatiquement

Fail2Ban lit les journaux et bloque temporairement les adresses IP qui échouent plusieurs fois à se connecter.

```
sudo apt install fail2ban -y
sudo nano /etc/fail2ban/jail.local

[sshd]
enabled  = true
port     = 4422
filter   = sshd
logpath  = /var/log/auth.log
maxretry = 3
findtime = 10m
bantime  = 1h

sudo systemctl restart fail2ban
sudo fail2ban-client status sshd
```

Ici, une IP qui échoue 3 fois en 10 minutes est bannie pendant 1 heure. Sur les systèmes sans `auth.log`, remplace `logpath` par `backend = systemd`.

2.7 Pare-feu UFW

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 4422/tcp                                          # AVANT d'activer le pare-feu
sudo ufw allow from 192.168.1.50 to any port 4422 proto tcp      # option : une seule IP source
```

```
sudo ufw enable
sudo ufw status verbose
```

2.8 Lire les journaux

```
sudo tail -f /var/log/auth.log | grep sshd      # Debian, Ubuntu, Kali
sudo journalctl -u ssh -f                       # systèmes avec journald
```

2.9 Donner accès à plusieurs utilisateurs

Règle : un utilisateur = un compte. On peut regrouper les comptes autorisés dans un groupe.

```
sudo adduser colle1
sudo adduser colle2
sudo groupadd sshusers
sudo usermod -aG sshusers colle1
sudo usermod -aG sshusers colle2
```

Puis dans `sshd_config` : `AllowUsers colle1 colle2` ou `AllowGroups sshusers`. Chaque personne génère sa clé (`ssh-keygen`) et l'envoie avec `ssh-copy-id colle1@IP`. Après modification : `sudo systemctl restart ssh`.

2.10 Erreurs fréquentes

Message	Cause probable	Solution
`Permission denied (publickey)`	Mot de passe coupé mais clé absente ou droits trop ouverts.	Vérifier la clé dans *authorized_keys* et les droits 700 (*~/.ssh*) et 600 (*authorized_keys*).
`Connection refused`	SSH n'écoute pas sur ce port, ou un pare-feu bloque.	Vérifier avec *ss -lnpt* et les règles UFW / iptables.
`Connection timed out`	Problème de routage ou pare-feu qui ignore les paquets (DROP).	Tester avec *nc -zv IP PORT* et vérifier les pare-feux.

2.11 Travaux pratiques : durcir une VM Kali

Objectif : sécuriser SSH sur une machine virtuelle Kali Linux, puis valider le résultat depuis une machine cliente.

- **Préparer les comptes** : `sudo adduser secuser`, puis `sudo usermod -aG sudo secuser`.
- **Générer et envoyer la clé** depuis le client : `ssh-keygen -t ed25519`, puis `ssh-copy-id secuser@IP_KALI`. Tester la connexion.
- **Durcir sshd_config** : port 4422, `PermitRootLogin no`, `PasswordAuthentication no`, `AllowUsers secuser`, inactivité de 3 minutes. Valider avec `sudo sshd -t`, puis redémarrer.
- **Filtrer** : UFW bloque tout sauf 4422/tcp ; Fail2Ban surveille le port 4422 (3 échecs = bannissement de 30 minutes).
- **Auditer** : depuis le client, `ssh root@IP_KALI -p 4422` doit être refusé ; `ssh kali@IP_KALI -p 4422` aussi (compte absent de *AllowUsers*) ; les tentatives apparaissent dans `/var/log/auth.log`.

3. SSH sous Windows (OpenSSH)

OpenSSH est intégré à Windows 10/11 et à Windows Server 2019/2022. Le serveur (`sshd`) est une fonctionnalité optionnelle. Toutes les commandes se lancent dans un terminal **administrateur**. Chaque étape est donnée en PowerShell et en CMD.

3.1 Installer et démarrer le serveur

PowerShell

```
Get-WindowsCapability -Online | Where-Object Name -Like 'OpenSSH*'      # vérifier
Add-WindowsCapability -Online -Name "OpenSSH.Server~~~~0.0.1.0"         # installer
Set-Service -Name sshd -StartupType Automatic                          # démarrage auto
Start-Service -Name sshd
Get-Service -Name sshd                                                 # état
Get-NetTCPConnection -LocalPort 22 -State Listen                       # port en écoute
```

CMD (sans PowerShell)

```
dism /online /get-capabilities | findstr /I "OpenSSH"
dism /online /add-capability /capabilityname:OpenSSH.Server~~~~0.0.1.0
sc config sshd start= auto            (l'espace après "=" est obligatoire)
net start sshd
sc query sshd                         (doit afficher STATE : 4 RUNNING)
netstat -ano | findstr :22
ipconfig                              (pour connaître l'adresse IP du PC)
```

Test depuis un autre appareil : `ssh utilisateur@IP_DU_PC`. Pour arrêter ou redémarrer : `net stop sshd` puis `net start sshd`.

3.2 Fichier de configuration

Le fichier est `C:\ProgramData\ssh\sshd_config`. Une ligne qui commence par **#** est un commentaire : le serveur utilise alors sa valeur par défaut.

```
notepad C:\ProgramData\ssh\sshd_config

:: Voir la configuration réellement appliquée
"C:\Windows\System32\OpenSSH\sshd.exe" -T | findstr /i "passwordauthentication pubkeyauthentication"
```

> **À savoir :** Sous Windows, il n'y a pas de compte *root* : `PermitRootLogin` n'a pas d'effet utile. On bloque le compte administrateur avec `DenyUsers`. Après chaque modification, redémarre le service.

3.3 Authentification par clé sous Windows

Type de compte	Où placer la clé publique
Utilisateur standard	`C:\Users\<utilisateur>\.ssh\authorized_keys`
Membre du groupe Administrateurs	`C:\ProgramData\ssh\administrators_authorized_keys`

OpenSSH rejette la clé si les permissions (ACL) du fichier sont trop larges. Pour le fichier des administrateurs :

```
:: CMD
echo ssh-ed25519 AAAAC3Nza... admin@company >> C:\ProgramData\ssh\administrators_authorized_keys
icacls.exe "C:\ProgramData\ssh\administrators_authorized_keys" /inheritance:r
icacls.exe "C:\ProgramData\ssh\administrators_authorized_keys" /grant:r "Administrators:(F)"
```

```
icacls.exe "C:\ProgramData\ssh\administrators_authorized_keys" /grant:r "NT AUTHORITY\SYSTEM:(F)"
```

`/inheritance:r` supprime l'héritage des droits ; seuls le groupe Administrateurs et le compte SYSTEM gardent le contrôle total. Connexion de test : `ssh -i ~\.ssh\id_ed25519 utilisateur@IP`.

3.4 Durcir la configuration

```
Port 45222
PubkeyAuthentication yes
PasswordAuthentication no
PermitEmptyPasswords no
MaxAuthTries 2
LoginGraceTime 30
AllowGroups SecOps_Admins
DenyUsers Administrateur Administrator
LogLevel VERBOSE
ClientAliveInterval 300
ClientAliveCountMax 2
```

Pour créer le groupe autorisé : `net localgroup SecOps_Admins /add`, puis `net localgroup SecOps_Admins NomUtilisateur /add`. `LogLevel VERBOSE` enregistre aussi l'empreinte de chaque clé présentée, utile pour l'audit.

> **Attention :** Ajoute `PasswordAuthentication no` seulement après avoir confirmé qu'une connexion par clé fonctionne.

3.5 Pare-feu Windows

PowerShell

```
Disable-NetFirewallRule -DisplayName "OpenSSH SSH Server (sshd)"
New-NetFirewallRule -Name "Prod_SSH_Sec" -DisplayName "SSH Securise (45222)" `
  -Direction Inbound -Action Allow -Protocol TCP -LocalPort 45222
Set-NetFirewallRule -Name "Prod_SSH_Sec" -RemoteAddress "10.0.10.0/24"   # limiter aux IP d'admin
```

CMD

```
netsh advfirewall firewall set rule name="OpenSSH SSH Server (sshd)" new enable=no
netsh advfirewall firewall add rule name="Prod_SSH_Sec" dir=in action=allow protocol=TCP localport=45222
netsh advfirewall firewall set rule name="Prod_SSH_Sec" new remoteip=10.0.10.0/24
```

3.6 Surveillance et audit

Les événements d'OpenSSH sont dans l'Observateur d'événements : *Journaux des applications et des services, puis OpenSSH, puis Operational*.

```
Get-WinEvent -LogName 'OpenSSH/Operational' -MaxEvents 20 | Select-Object TimeCreated, Message

:: CMD
wevtutil qe OpenSSH/Operational /c:20 /rd:true /f:text
```

3.7 Dépannage

- Le serveur demande encore le mot de passe malgré la clé : vérifier les droits avec `icacls` sur le fichier de clés.

- Vérifier en fin de *sshd_config* la présence du bloc `Match Group administrators` pour les comptes administrateurs.
- Enregistrer le fichier en **UTF-8** et valider la configuration avec `sshd.exe -T`.
- Le service ne démarre plus après une modification : annuler la dernière ligne ajoutée et relancer `net start sshd`.

4. Transférer des fichiers avec SSH

> **Règle d'or : SCP** se lance toujours depuis le **PC local** (CMD Windows ou terminal Linux), sans se connecter d'abord en SSH. **SFTP** ouvre une session interactive depuis le PC local.

4.1 SCP (copie en une ligne)

```
scp [fichier_local] [utilisateur]@[IP]:[destination_distante]        # envoyer
scp [utilisateur]@[IP]:[fichier_distant] [destination_locale]        # télécharger
scp -r dossier_local/ utilisateur@IP:/destination                    # dossier entier (-r)
scp -P 4422 fichier.txt utilisateur@IP:/home/utilisateur/            # port SSH personnalisé (P majuscule
```

Exemples avec des valeurs réelles

```
:: Windows (CMD) vers Linux
scp "C:\Users\Admin\rapport.pdf" ubuntu@192.168.1.50:/home/ubuntu/

# Linux vers Linux
scp ~/rapport.pdf ubuntu@192.168.1.50:/home/ubuntu/

# Télécharger depuis le serveur vers le dossier courant (le point = ici)
scp ubuntu@192.168.1.50:/var/www/html/index.html .
```

4.2 SFTP (session interactive)

```
sftp ubuntu@192.168.1.50
```

Commande	Rôle	Exemple
`ls`	Lister le dossier distant	`ls`
`lls`	Lister le dossier local	`lls`
`cd`	Changer de dossier distant	`cd /etc/nginx`
`put`	Envoyer un fichier local vers le serveur	`put photo.jpg`
`get`	Télécharger un fichier du serveur	`get document.pdf`
`exit`	Fermer la session SFTP	`exit`

4.3 Scénario complet

```
:: 1. Envoyer une archive de Windows vers le serveur (depuis CMD, sans "ssh" avant)
scp "C:\Projets\site.zip" debian@10.0.0.22:/var/tmp/

:: 2. Récupérer un fichier de configuration avec SFTP
cd C:\Users\Admin\Desktop
sftp debian@10.0.0.22
sftp> cd /etc/nginx
sftp> ls
sftp> get nginx.conf
sftp> exit
```

5. Aide-mémoire et bonnes pratiques

Objectif	Directive sshd_config	Valeur recommandée
Changer le port	`Port`	Hors 22 (ex. 4422, 45222)
Interdire root (Linux)	`PermitRootLogin`	`no`
Interdire l'admin (Windows)	`DenyUsers`	`Administrateur Administrator`
Désactiver les mots de passe	`PasswordAuthentication`	`no`
Activer les clés	`PubkeyAuthentication`	`yes`
Comptes vides interdits	`PermitEmptyPasswords`	`no`
Liste blanche	`AllowUsers / AllowGroups`	Comptes ou groupes précis
Limiter les essais	`MaxAuthTries`	2 ou 3
Délai d'authentification	`LoginGraceTime`	30
Déconnexion si inactif	`ClientAliveInterval / CountMax`	par ex. 90 et 1
Pas de redirection graphique	`X11Forwarding`	`no`
Journal détaillé	`LogLevel`	`VERBOSE`

Checklist avant de terminer

- Sauvegarde de *sshd_config* faite.
- Connexion par clé testée **avant** de couper les mots de passe.
- Syntaxe validée (`sshd -t` sous Linux, `sshd.exe -T` sous Windows).
- Port ouvert dans le pare-feu et déclaré dans Fail2Ban.
- Une seconde session de test ouverte avant de fermer la première.
- Système et OpenSSH mis à jour régulièrement (`sudo apt update && sudo apt upgrade`).
- Clés privées protégées par une phrase de passe, jamais partagées ni publiées sur GitHub.

Glossaire

Terme	Définition
ACL	Liste de contrôle d'accès : droits donnés à des utilisateurs sur un fichier ou un dossier.
Force brute	Attaque qui teste automatiquement de nombreux mots de passe.
Ed25519	Algorithme de signature moderne, recommandé pour les clés SSH.
Fail2Ban	Outil qui bannit les adresses IP après plusieurs échecs de connexion.
Hardening	Durcissement : réduire la surface d'attaque d'un système.
MITM	Attaque de l'homme du milieu : un tiers s'intercale entre le client et le serveur.

> **Éthique :** Ces techniques s'appliquent à mes propres machines et à mon laboratoire isolé. Tester ou attaquer un système sans autorisation est illégal.